

HUMAIN



HUMAIN Create Studio

PREPARED BY  A A V Y A

Production Readiness & Security Assessment

Pre-deployment validation — application, security, dependencies, architecture

System	HUMAIN Create Studio (Payload v3 CMS + Studio console + AI service)
Environment assessed	Live single-VM deployment (<code>cms.34-14-150-134.sslip.io</code>)
Prepared by	Aavya
Prepared for	HUMAIN / Engineering & IT Security
Classification	Confidential
Date	June 2026

1. Executive summary

The **application layer is in good shape**: authentication, RBAC/ABAC, user management and core workflows were validated end-to-end against the live system and pass cleanly, and the public attack surface is well-controlled (no Payload admin/REST or `.env` exposure, strong HTTP security headers, secure cookies, parameterised DB queries).

However, "**production-grade, handles any load, zero glitches**" is not yet a claim that can be made — and that is an **architecture/operations** gap, not an application-code gap. The system currently runs on a **single shared VM** with no redundancy, autoscaling, managed-DB failover or formal DR, alongside other

live products. In addition, this assessment found a small number of **application hardening items** (cross-user data segregation, login brute-force protection, dependency CVEs) that should be remediated before a production cutover.

Verdict: *Application is solid and safe for controlled/pilot production. For "any-load, mission-critical" production, complete the architecture hardening (Section 5) and the P0/P1 fixes (Section 7), then run load testing in an isolated environment (Section 6).*

2. Scope and method

Tested **live** via the console's own APIs (what the UI calls), the Payload REST layer, and a real headless browser:

- **Security / vulnerability assessment** — auth & session, access-control bypass, IDOR, security headers, public-surface exposure, secret leakage.
- **Dependency / CVE scan** — `pnpm audit` across `cms`, `console`, `ai-service`.
- **Integration / E2E** — 28 console-API checks, 17 browser-UI checks, 36 RBAC/ABAC behavioural checks, multi-role assignment.
- **Production-readiness review** — architecture, scaling, data durability, observability.
- **Load testing** — *plan only*; to be executed in an isolated environment (Section 6) to avoid impacting co-hosted live products.

3. What passed (validated, no issues)

Area	Result
Authentication & sessions	Login/logout work; wrong password rejected; cookie is Secure; HttpOnly; SameSite=Lax
Authorisation (RBAC + ABAC)	36/36 behavioural checks — role gates, site/department/locale scope, field-level, publish gating, no privilege escalation
User management	Create / edit / multi-role assign / activate-deactivate / delete; non-admins blocked (403); creation-date shown
Public attack surface	<code>/admin</code> 404 , raw Payload REST not exposed , <code>/.env</code> 404 ; protected APIs return 401/403 unauthenticated
HTTP security headers	HSTS, CSP , <code>X-Frame-Options: SAMEORIGIN</code> , <code>X-Content-Type-Options: nosniff</code> , Referrer-Policy, Permissions-Policy
Injection resistance	Payload + Drizzle parameterised queries (strong against SQL injection)
Rate limiting (AI service)	<code>@fastify/rate-limit</code> registered on the generation service
Functional / integration	28/28 API + 17/17 UI checks; graceful error handling (e.g. AI credit exhaustion streams a clean error, no crash)

4. Findings and recommendations

#	Severity	Finding	Recommendation
F1	Medium	Cross-user data segregation (IDOR). Projects (and non-archetype Brand Guidelines) are role-scoped, not owner-scoped — any <i>editor</i> can list and modify another user's projects (verified: an author edited an admin's project). Viewers are correctly excluded.	Add owner-scoped access on Projects/BrandGuidelines (<code>owner == user.id</code> OR admin), and filter the list endpoints by owner. Decide explicitly if projects are meant to be private or team-shared.
F2	Medium	No login brute-force protection. The Payload auth collection has no <code>maxLoginAttempts</code> / <code>lockTime</code> , and the login path is not rate-limited.	Enable Payload account logout (<code>maxLoginAttempts</code> , <code>lockTime</code>) and add edge rate-limiting on <code>/api/auth/login</code> (nginx or WAF).
F3	Medium–High	Dependency CVEs — <code>pnpm audit</code> : 2 critical · 12 high · 26 moderate · 5 low. Most are dev-only (vitest, esbuild, vite — not shipped) or admin-UI-only (dompurify via monaco, <code>/admin</code> not exposed). Runtime-relevant : <code>fastify</code> / <code>fast-uri</code> (AI service), <code>protobufjs</code> (via embeddings).	Run <code>pnpm update</code> + add <code>overrides</code> / <code>resolutions</code> to pin patched versions (esp. fastify → latest 4.x, <code>protobufjs</code>); re-audit. Add SCA scanning (Dependabot/Snyk) to CI.
F4	Low	Version disclosure — <code>X-Powered-By: Next.js</code> header present.	Disable <code>poweredByHeader</code> in Next config (defence-in-depth).
F5	Low	CSP allows 'unsafe-inline' for scripts/styles.	Tighten toward nonces/hashes where practical.
F6	Medium	Search latency under load — <code>/api/search</code> p99 ~6 s at 25 concurrent (Section 6.1).	Add caching, optimise the vector/text query, and scale horizontally before high-concurrency search workloads.

No critical application vulnerabilities (auth bypass, unauth data access, injection, secret exposure) were found.

Remediation status (this engagement): F1, F2 and F3 have been implemented, deployed and re-verified on the live VM — projects are now owner-private (IDOR closed), login logout is active (account locks after 5 failed attempts; verified), and runtime dependency CVEs were patched (audit 45 → 34, runtime-relevant cleared). **F4–F6** remain open (low/medium, non-blocking).

5. Production-readiness (architecture) gaps

These determine whether the system can be relied on under real production load. They are **infrastructure/ops**, not bugs:

Gap	Risk	Recommendation
Single shared VM — no redundancy; co-hosted with other live products; a reboot or noisy neighbour = full outage	High	Move to separate, isolated Prod with ≥ 2 app instances behind a load balancer; isolate from other products (the Dev/Staging/Prod model).
DB schema via dev-push, not migrations	Medium–High	Adopt generated Payload migrations ; gate schema changes through CI.
No documented managed backups / PITR / DR	High	Managed Postgres with automated backups + point-in-time recovery; tested restore runbook.
No autoscaling; capacity unverified	Medium	Establish baseline via load testing (Section 6); add horizontal autoscaling.
Observability — no centralised metrics/alerting/APM confirmed	Medium	Add metrics + alerting (latency, error rate, saturation) and structured log aggregation.
Secrets in <code>.env.production</code> on host	Medium	Move to a managed secrets manager; rotate <code>PAYLOAD_SECRET</code> and provider keys.
No WAF / edge rate-limiting	Medium	Front with a WAF; rate-limit auth and generation endpoints at the edge.

6. Load & capacity testing plan (isolated environment)

Load testing will be run against an **isolated environment** so the co-hosted live products are never at risk. Plan:

- Provision isolation** — a dedicated VM (or a local Docker Compose stack from the same images) mirroring the Prod topology; seed representative data.
- Scenarios** — (a) read-heavy public content delivery, (b) authenticated Studio/CMS CRUD, (c) AI generation concurrency (requires Anthropic credit), (d) soak test (sustained load over time).
- Tooling** — `k6` / Artillery; ramp virtual users; measure p50/p95/p99 latency, throughput, error rate, and resource saturation (CPU/RAM/DB connections).
- Outputs** — capacity baseline (max sustainable RPS/concurrency), breaking point, and right-sizing recommendations feeding the sizing matrix.

Note: meaningful AI-generation load testing is **blocked until Anthropic credits are restored**; non-AI paths can be tested immediately.

6.1 Initial controlled results (on the live VM)

A controlled test (25 concurrent connections $\times$ 12 s per scenario, run on the VM with resource monitoring) confirmed the system is **stable under sustained concurrency — zero errors, zero non-2xx, no crashes or connection-pool exhaustion** — while the host load average barely moved (co-hosted products unaffected).

Scenario	Throughput	p50	p95	p99	Errors
Public SSR <code>/login</code>	~84 req/s	279 ms	381 ms	802 ms	0
Auth API <code>/api/projects</code>	~37 req/s	647 ms	980 ms	1.28 s	0
Auth API <code>/api/notifications</code>	~39 req/s	623 ms	722 ms	801 ms	0
Auth API <code>/api/search</code>	~14 req/s	1.67 s	4.24 s	6.19 s	0

Reading: no stability problems, but **modest throughput** (single shared VM) and **search is a latency hotspot** that degrades sharply under load (p99 ~6 s). For higher concurrency: horizontal scaling, DB connection pooling/read replicas, and **search optimisation/caching** are the levers. These are capacity/perf items, not defects.

7. Prioritised remediation roadmap

P0 — before production cutover

- ✅ **Done:** F1 owner-scope Projects/BrandGuidelines · F2 login logout · F3 patch runtime deps (fastify/protobuffs) — *implemented, deployed and verified*
- Remaining: isolated **Prod** environment + managed Postgres with **backups/PITR** · Payload **migrations** · edge rate-limit on login

P1 — at/with launch

- Autoscaling + load-test-driven sizing · monitoring/alerting + log aggregation · WAF + edge rate-limiting · secrets manager

P2 — hardening / continuous

- CSP tightening · remove version headers · SCA + dependency scanning in CI · DR drills · MFA for admin accounts

8. Conclusion

The product is **functionally complete and the application security posture is strong** — no critical application vulnerabilities, validated access control, and a well-contained public surface. It is **suitable for controlled/pilot production today**. To support **full production at scale with confidence**, complete the P0 architecture and hardening items, then validate capacity via isolated load testing. None of the open items are blocking defects in the code; they are the expected steps to take a working application to a resilient, production-grade service.

Prepared by Aavya for HUMAIN — Confidential. Production readiness & security assessment of HUMAIN Create Studio, based on live validation of the current deployment.